



Universidad Veracruzana
Facultad de Instrumentación electrónica

Reporte sobre:
Graficos con phyton

Asignatura:
Programación Avanzada

Docente de la asignatura:
Domínguez Chávez José Alfonso

Alumno:
Nadia Fernanda Barradas Solís
Josue Muñoz Camacho
Martines Morales Luis Antonio
Eduardo Acosta Lopez
Ivon Gonzales Avendano
Daniel Alberto Gil Martinez



Tabla de contenido

1- Cargar de datos	1
2- Aplicación de promediado Movil.....	4
3- Aplicación de Filtros Pasa bandas.....	6
4- Aplicación de Filtros Pasa bajas.....	8



1- Código cargar datos

Este código en Python realiza lo siguiente:

1. Importa bibliotecas necesarias:

- pandas: para leer y manejar archivos CSV con datos tabulados.
- matplotlib.pyplot: para generar gráficos.

2. Carga datos meteorológicos desde tres archivos CSV:

- temperatura.csv → datos de temperatura.
- humedad.csv → datos de humedad relativa.
- viento.csv → datos de velocidad y dirección del viento.

3. Crea una figura con cuatro subgráficos verticales (usando plt.subplot), cada uno mostrando una señal diferente en función del tiempo (excepto uno):

- **Subgráfico 1:** Temperatura vs. Tiempo.
- **Subgráfico 2:** Humedad relativa vs. Tiempo.
- **Subgráfico 3:** Velocidad del viento vs. Tiempo.
- **Subgráfico 4:** Dirección del viento (no usa tiempo como eje X).

De igual manera Agrega títulos, etiquetas y rejillas a cada gráfico para mejor visualización., Ajusta el espaciado entre los subgráficos para evitar que se solapen. Y Muestra los gráficos todos juntos en una sola ventana.

• Imágenes del código:

```
1 #Version 1.1.0
2 # Importación de bibliotecas necesarias
3 import pandas as pd # Para manejo de datos y DataFrames
4 import matplotlib.pyplot as plt # Para visualización gráfica
5
6 # ----- CARGA DE DATOS -----
7 # Cargar los archivos CSV usando pandas
8 # Se especifica separador decimal como punto y delimitador de columnas como coma
9 temperatura = pd.read_csv('temperatura.csv', sep=',', decimal= '.')
10 humedad = pd.read_csv('humedad.csv', sep=',', decimal= '.')
11 viento = pd.read_csv('viento.csv', sep=',', decimal= '.')
12
13 # ----- CONFIGURACIÓN DE GRÁFICOS -----
14 # Primer subgráfico: Temperatura
15 plt.subplot(4, 1, 1) # (filas, columnas, posición)
16 plt.plot(temperatura["Tiempo"], temperatura["Temperatura_C"] ) # (x, y)
17 plt.title("Señal original - Temperatura") # Título de la grafica
18 plt.ylabel("Temperatura - Centigrados") # Nombre de la grafica Y
19 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
20
21 # Segundo subgráfico: Humedad
22 plt.subplot(4,1,2)
23 plt.plot(humedad["Tiempo"], humedad ["Humedad_Relativa_%"]) # (x, y)
24 plt.title("Señal original - Humedad") # Título de la grafica
25 plt.ylabel("Humedad - Relativa") # Nombre de la grafica Y
26 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
27
```



```
# Tercer subgráfico: Velocidad del viento
plt.subplot(4,1,3) # (filas, columnas, posición)
plt.plot(viento["Tiempo"], viento["Velocidad_Viento_mps"]) # (x, y)
plt.title("Señal original - Viento") # Título de la grafica
plt.ylabel("Velocidad - Mps") # Nombre de la grafica Y
plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.

# Cuarto subgráfico: Dirección del viento
plt.subplot(4,1,4)
plt.plot(viento["Direccion_Viento_deg"]) # (x, y)
plt.title("Señal original - Viento") # Título de la grafica
plt.ylabel("Dirección - DEG") # Nombre de la grafica Y
plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.

# Ajustar espaciado entre subgráficos para evitar superposiciones
plt.tight_layout()

# Mostrar la figura con todos los subgráficos
plt.show()
```

2- Aplicar promediado móvil

El propósito de este código es **leer datos meteorológicos**, aplicarles un **suavizado mediante promedio móvil** y **visualizar tanto los datos originales como los suavizados** en gráficos. Las variables representadas son: temperatura, humedad relativa, velocidad del viento y dirección del viento.

Utiliza las bibliotecas pandas para manejar los datos y matplotlib.pyplot para graficarlos. Primero, se leen los datos desde archivos CSV y se calcula un tiempo artificial multiplicando el índice de los datos por 5 (posiblemente para simular que las mediciones se hicieron cada 5 unidades de tiempo).

Luego, se aplica un **promedio móvil de 3 puntos** a cada una de las variables para suavizar las señales y hacer más visibles las tendencias. A continuación, se generan cuatro gráficos dispuestos verticalmente: el primero muestra la temperatura original y su versión suavizada, el segundo hace lo mismo con la humedad relativa, el tercero con la velocidad del viento, y el cuarto con la dirección del viento.

En cada gráfico se incluyen colores distintos, líneas punteadas para el promedio móvil, leyendas, títulos, etiquetas de eje y una rejilla de fondo para facilitar la interpretación. Finalmente, se ajusta el espaciado entre los subgráficos y se muestra toda la figura en pantalla.



- Imágenes del código:

```
1  #Version 2.1.2
2  # Importación de bibliotecas necesarias.
3  import pandas as pd # Para manejo de datos y DataFrames.
4  import matplotlib.pyplot as plt # Para visualización gráfica.
5
6  # ----- CARGA DE DATOS -----
7  # Cargar los archivos CSV usando pandas.
8  # Se especifica separador decimal como punto y delimitador de columnas como coma.
9  temperatura = pd.read_csv('temperatura.csv', sep= ",", decimal= ".")
10 humedad = pd.read_csv('humedad.csv', sep= ",", decimal= ".")
11 viento = pd.read_csv('viento.csv', sep= ",", decimal= ".")
12
13 def tiempo (df): # Calcula el tiempo transcurrido basado en el índice del DataFrame.
14     return df.index * 5 # Multiplica cada valor del índice por 5 para obtener el tiempo.
15
16 # Calcula el promedio móvil de 3 puntos para la temperatura
17 # rolling(3): Usa una ventana de 3 periodos (valores consecutivos)
18 # mean(): Calcula el promedio dentro de la ventana
19 temperatura["Promediado movil"] = temperatura["Temperatura_C"].rolling(3).mean()
20
21 # Aplica el mismo promedio móvil a la humedad relativa
22 humedad["Promediado movil"] = humedad["Humedad_Relativa_%"].rolling(3).mean()
23
24 # Calcula promedio móvil para velocidad del viento (nueva columna PM_Velocidad)
25 viento["PM_Velocidad"] = viento["Velocidad_Viento_mps"].rolling(3).mean()
26
27 # Calcula promedio móvil para dirección del viento (nueva columna PM_Direccion)
28 viento["PM_Direccion"] = viento["Direccion_Viento_deg"].rolling(3).mean()
29
```

```
30 # ----- CONFIGURACIÓN DE GRÁFICOS -----
31 # Crear una figura con 4 subgráficos en disposición vertical (4 filas, 1 columna)
32 plt.figure(figsize=(10, 12)) # Tamaño opcional para mejor visualización.
33
34 # Primer subgráfico: Temperatura
35 plt.subplot(4,1,1) # (filas, columnas, posición)
36 plt.plot(tiempo(temperatura),temperatura["Temperatura_C"], color = "orange", label = "Original") # (x, y), ademas de el color y la leyenda de la grafica.
37 plt.plot(tiempo(temperatura), temperatura["Promediado movil"], color = "blue", linestyle = "--", label = "Promediado") # (x, y), ademas de el color, la leyenda
38 plt.title("Señal - Temperatura") # Título de la grafica
39 plt.ylabel("Temperatura - °C") # Nombre de la grafica Y
40 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
41 plt.legend() # Mostrar la leyenda de cada grafica.
42
43 plt.subplot(4,1,2) # (filas, columnas, posición)
44 plt.plot(tiempo(humedad), humedad ["Humedad_Relativa_%"], color = "green", label = "Original") # (x, y), ademas de el color y la leyenda de la grafica.
45 plt.plot(tiempo(humedad), humedad["Promediado movil"], color = "red", linestyle = "--", label = "Promediado") # (x, y), ademas de el color, la leyenda de
46
47 plt.title("Señal - Humedad") # Título de la grafica
48 plt.ylabel("Humedad - %") # Nombre de la grafica Y
49 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
50 plt.legend() # Mostrar la leyenda de cada grafica.
51
52 plt.subplot(4,1,3) # (filas, columnas, posición)
53 plt.plot(tiempo(viento), viento["Velocidad_Viento_mps"], label = "Original") # (x, y), ademas de la leyenda de la grafica.
54 plt.plot(tiempo(viento), viento["PM_Velocidad"], color = "orange", linestyle = "--", label = "Promediado") # (x, y), ademas de el color, la leyenda de la graf
55
56 plt.title("Señal - Viento") # Título de la grafica
57 plt.ylabel("Velocidad - m/s") # Nombre de la grafica Y
58 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
59 plt.legend() # Mostrar la leyenda de cada grafica.
```



```
1 plt.subplot(4,1,4) # (filas, columnas, posición)
2 plt.plot(tiempo(viento), viento["Direccion_Viento_deg"], color = "grey", label = "Original") # (x, y), ademas de el color y la leyenda de la grafica.
3 plt.plot(tiempo(viento), viento["PM_Direccion"], color = "purple", linestyle = "--", label = "Promediado") # (x, y), ademas de el color, la leyenda de la graf
4
5 plt.title("Señal - Viento/Dirección") # Título de la grafica
6 plt.ylabel("Dirección - DEG") # Nombre de la grafica Y
7 plt.grid() # Rejilla en la grafica para no mostrar un fondo blanco.
8 plt.legend() # Mostrar la leyenda de cada grafica.
9
10 # Ajustar espaciado entre subgráficos para evitar superposiciones
11 plt.tight_layout()
12
13 # Mostrar la figura con todos los subgráficos
14 plt.show()
```

3- Aplicar Filtro Pasa Bandas

Este código en Python tiene como objetivo aplicar un **filtro pasa bandas** a señales contenidas en archivos CSV (por ejemplo, datos de temperatura, humedad o viento), y comparar visualmente la señal original con la señal filtrada. Utiliza las bibliotecas `pandas` para manejar datos, `matplotlib` para graficarlos, `numpy` para operaciones numéricas, y `scipy.signal` para aplicar el filtro digital.

Primero, configura el estilo de las gráficas usando un tema tipo `ggplot` y establece un tamaño de figura amplio. Luego define una función llamada `aplicar_filtro_pasa_bandas`, que crea un filtro digital tipo **Butterworth** de orden 4 y lo aplica a una señal. Este filtro deja pasar solo las frecuencias entre 0.1 Hz y 0.3 Hz, eliminando el resto. Para ello, convierte las frecuencias a proporciones respecto al **teorema de Nyquist** (la mitad de la frecuencia de muestreo) y usa `filtfilt`, que aplica el filtro hacia adelante y hacia atrás para evitar desfases.

Después, otra función `cargar_y_procesar_csv` se encarga de leer un archivo CSV (suponiendo que la primera columna es el tiempo y la segunda son los datos), aplicar el filtro pasa bandas, y devolver tanto los datos originales como los filtrados.

En la parte principal del código, se definen los nombres de los archivos CSV a procesar (humedad, temperatura y viento) y se establece una frecuencia de muestreo (`fs`) de 10 Hz, lo cual indica que hay 10 datos por segundo. Luego, se abre una sola figura para graficar y se recorren los archivos uno por uno. Para cada archivo, se cargan los datos, se filtran y se grafican ambos: los datos originales (línea continua semitransparente) y los filtrados (línea discontinua más marcada), usando distintos colores para cada archivo.

Finalmente, se configura la gráfica general con título, etiquetas, leyenda externa (fuera del gráfico), rejilla y un diseño ajustado con `tight_layout()` para evitar superposición de elementos. Al terminar, se muestra la figura con `plt.show()`.



- Imágenes del código

```
1  #código del filtro pasa bandas parte final
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import numpy as np
5  from scipy import signal
6
7  # Configuración de estilo para las gráficas
8  plt.style.use('ggplot')
9  plt.rcParams['figure.figsize'] = (12, 8)
10
11  def aplicar_filtro_pasa_bandas(datos, fs, lowcut_hz=0.1, highcut_hz=0.3):
12      """Aplica un filtro pasa bandas con frecuencias en Hz"""
13      nyquist = 0.5 * fs
14      low = lowcut_hz / nyquist
15      high = highcut_hz / nyquist
16      b, a = signal.butter(4, [low, high], btype='band')
17      return signal.filtfilt(b, a, datos)
18
19  def cargar_y_procesar_csv(archivo_csv, fs):
20      """Carga un archivo CSV y devuelve los datos originales y filtrados"""
21      df = pd.read_csv(archivo_csv)
22      tiempo = df.iloc[:, 0].values
23      datos_originales = df.iloc[:, 1].values
24
25      # Aplicar filtro pasa bandas
26      datos_filtrados = aplicar_filtro_pasa_bandas(datos_originales, fs=fs)
27
28      return tiempo, datos_originales, datos_filtrados
29
30  # Archivos CSV a leer (cambiar por tus rutas reales)
31  archivos_csv = ['humedad.csv', 'temperatura.csv', 'viento.csv']
32
```

```
def cargar_y_procesar_csv(archivo_csv, fs):
    # Frecuencia de muestreo (¡IMPORTANTE! ajusta este valor según tus datos)
    fs = 10 # Ejemplo: 10 Hz (muestras por segundo)

    # Crear una sola figura
    plt.figure()

    # Colores diferenciados para cada archivo
    colores = ['b', 'g', 'r', 'c', 'm', 'y']

    # Procesar cada archivo CSV y agregar a la gráfica
    for i, archivo in enumerate(archivos_csv):
        try:
            tiempo, original, filtrado = cargar_y_procesar_csv(archivo, fs)

            # Graficar datos originales (línea continua)
            plt.plot(tiempo, original,
                     color=colores[i%len(colores)],
                     linestyle='-',
                     alpha=0.5,
                     label=f'{archivo} - Original')
```



```
54         # Graficar datos filtrados (línea discontinua)
55         plt.plot(tiempo, filtrado,
56                  color=colores[i%len(colores)],
57                  linestyle='--',
58                  linewidth=1.5,
59                  label=f'{archivo} - Filtrado (0.1-0.3 Hz)')
60
61     except Exception as e:
62         print(f'Error procesando {archivo}: {str(e)}')
63
64     # Configuración de la gráfica
65     plt.title(f'Comparación de datos originales y filtrados (fs={fs} Hz)')
66     plt.xlabel('Tiempo (s)')
67     plt.ylabel('Amplitud')
68     plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left') # Leyenda fuera del gráfico
69     plt.grid(True)
70     plt.tight_layout()
71
72     plt.show()
```

4- Aplicación del filtro pasa bajas:

Este código en Python compara tres formas de procesar señales ambientales (temperatura, humedad y viento): los **datos originales**, un **promedio móvil** y un **filtro pasa bajos**. Su propósito es visualizar cómo se suavizan y filtran las señales para eliminar el ruido o fluctuaciones bruscas.

1. Carga de datos

Se cargan tres archivos CSV que contienen datos de temperatura, humedad y viento. Cada uno se lee con pandas y se asume que tiene una columna de tiempo y otra con los valores medidos.

2. Creación de un eje de tiempo

La función tiempo(df) crea un eje temporal multiplicando el índice de cada fila por 5, suponiendo que las mediciones fueron tomadas cada 5 segundos.

3. Cálculo del promedio móvil

Se aplica un **promedio móvil centrado de 3 puntos** (rolling(3, center=True).mean()) a cada señal:

- Temperatura → Promediado_movil
- Humedad → Promediado_movil
- Viento → PM_Velocidad y PM_Direccion

Esto suaviza la señal conservando la forma general pero eliminando pequeños picos.

4. Aplicación de filtro pasa bajos

Se define la función aplicar_filtro, que implementa un **filtro digital pasa bajos Butterworth de orden 4**, con frecuencia de corte de 0.1 (valor normalizado). Este filtro elimina las frecuencias altas (ruido) manteniendo solo los cambios lentos en la señal. Se aplica a todas las señales, generando nuevas columnas:

- Temperatura → Filtro_PB
- Humedad → Filtro_PB
- Viento (velocidad y dirección) → Filt_Velocidad y Filt_Direccion



5. Visualización

Se genera una figura con 4 subgráficos (uno por señal), donde se comparan:

- La señal original (línea azul clara)
- El promedio móvil (línea naranja discontinua)
- El filtro pasa bajos (línea roja gruesa)

- Imágenes del código:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from scipy.signal import butter, filtfilt
4
5 # Cargar los archivos CSV
6 temperatura = pd.read_csv('temperatura.csv', sep=",", decimal=".")
7 humedad = pd.read_csv('humedad.csv', sep=",", decimal=".")
8 viento = pd.read_csv('viento.csv', sep=",", decimal=".")
9
10 def tiempo(df):
11     return df.index * 5
12
13 temperatura["Promediado_movil"] = temperatura["Temperatura_C"].rolling(3, center=True).mean()
14 humedad["Promediado_movil"] = humedad["Humedad_Relativa_%"].rolling(3, center=True).mean()
15 viento["PM_Velocidad"] = viento["Velocidad_Viento_mps"].rolling(3, center=True).mean()
16 viento["PM_Direccion"] = viento["Direccion_Viento_deg"].rolling(3, center=True).mean()
17
18
19 def aplicar_filtro(senal, fc=0.1, orden=4):
20     b, a = butter(orden, fc, btype='low')
21     return filtfilt(b, a, senal)
22
23 temperatura["Filtro_PB"] = aplicar_filtro(temperatura["Temperatura_C"])
24 humedad["Filtro_PB"] = aplicar_filtro(humedad["Humedad_Relativa_%"])
25 viento["Filt_Velocidad"] = aplicar_filtro(viento["Velocidad_Viento_mps"])
26 viento["Filt_Direccion"] = aplicar_filtro(viento["Direccion_Viento_deg"])
27
28 plt.figure(figsize=(12, 16))
```

```
30 v estilo = {
31     'original': {'color': 'navy', 'alpha': 0.6, 'linewidth': 1},
32     'movil': {'color': 'darkorange', 'linestyle': '--', 'linewidth': 1.5},
33     'filtro': {'color': 'crimson', 'linewidth': 2}
34 }
35
36 plt.subplot(4,1,1)
37 plt.plot(tiempo(temperatura), temperatura["Temperatura_C"], label="Original", **estilo['original'])
38 plt.plot(tiempo(temperatura), temperatura["Promediado_movil"], label="Prom. Móvil (V=3)", **estilo['movil'])
39 plt.plot(tiempo(temperatura), temperatura["Filtro_PB"], label="Filtro PB (fc=0.1)", **estilo['filtro'])
40 plt.title("Comparación: Temperatura")
41 plt.ylabel("Temperatura (°C)")
42 plt.grid(alpha=0.3)
43 plt.legend()
44
45 plt.subplot(4,1,2)
46 plt.plot(tiempo(humedad), humedad["Humedad_Relativa_%"], label="Original", **estilo['original'])
47 plt.plot(tiempo(humedad), humedad["Promediado_movil"], label="Prom. Móvil (V=3)", **estilo['movil'])
48 plt.plot(tiempo(humedad), humedad["Filtro_PB"], label="Filtro PB (fc=0.1)", **estilo['filtro'])
49 plt.title("Comparación: Humedad")
50 plt.ylabel("Humedad (%)")
51 plt.grid(alpha=0.3)
52 plt.legend()
```



```
54 plt.subplot(4,1,3)
55 plt.plot(tiempo(viento), viento["Velocidad_Viento_mps"], label="Original", **estilo['original'])
56 plt.plot(tiempo(viento), viento["PM_Velocidad"], label="Prom. Móvil (V=3)", **estilo['movil'])
57 plt.plot(tiempo(viento), viento["Filt_Velocidad"], label="Filtro PB (fc=0.1)", **estilo['filtro'])
58 plt.title("Comparación: Velocidad Viento")
59 plt.ylabel("Velocidad (m/s)")
60 plt.grid(alpha=0.3)
61 plt.legend()
62
63 plt.subplot(4,1,4)
64 plt.plot(tiempo(viento), viento["Direccion_Viento_deg"], label="Original", **estilo['original'])
65 plt.plot(tiempo(viento), viento["PM_Direccion"], label="Prom. Móvil (V=3)", **estilo['movil'])
66 plt.plot(tiempo(viento), viento["Filt_Direccion"], label="Filtro PB (fc=0.1)", **estilo['filtro'])
67 plt.title("Comparación: Dirección Viento")
68 plt.xlabel("Tiempo (segundos)")
69 plt.ylabel("Dirección (°)")
70 plt.grid(alpha=0.3)
71 plt.legend()
72
73 plt.tight_layout()
74 plt.show()
```